Burning_Gate: Binary Exploitation - Beginner

Creator: Eddie Points: –

Description

This challenge is a challenge that features a Cuckoo Filter data structure in action. The challenger has to insert the word ‘Flame’ in a specific order to spell it out through the Cuckoo Filter Buckets.

Github Path: Capture_the_Flame_2026/Binary_Exploit/FlameGate

Prompt

Oh No! You’ve stumbled upon a forest in your path and with no way around. The only way is to go through but you are met with a gate that won’t let you in without completing this puzzle.

<FlameGate.zip>

Hints

1. Try going into the code to see what the gate is looking for.
2. I wonder what is happening to the input...?
3. Set fire to the gate!

Solution

1. The competitor will have to try out the executable first, and see that it is shifting the letters. This is through the hash function that is implemented.

```
FlameGate$ ./FlameGate

Hello fair traveler.. This gate protects people from the terrors inside this forest.To get into the forest, you have to solve the puzzle of the gate to prove you are clever enough to not succumb to the
 forest and go mad. Try at your own risk..
Enter up to 10 characters (without spaces): Hello!


       [H]    [!]    [l]    [l]    [e]    [o]


       **Nothing happens to the gate as you finalize your answer**

Sorry...you're gonna have to try again... I cannot allow you to enter
FlameGate$
```

2. You then have to go into gdb to see the source code to build a sense of where to investigate. List the code in gdb to see the following

```
(gdb) l
1       #include "Filter.h"
2       #include <stdio.h>
3       #include <stdlib.h>
4       #include <string.h>
5
6
7       int main() {
8           printPrompt();
9           Queue* elements = getUserInput();
10
(gdb)
11          int n = elements->size;
12          int* B = createBuckets(n);
13
14          Shuffle(B, n, elements);
15          printSlots(B,n);
16
17          return 0;
18      }
(gdb)
```

3. The user will then have to investigate Shuffle and printSlots. Shuffle will provide info about the hash functions and the movement implementation, and printSlots will give you an idea of what the gate is looking for.

Shuffle:

```
Dump of assembler code for function Shuffle:
   0x00000000000014c3 <+0>:     endbr64
   0x00000000000014c7 <+4>:     push   %rbp
   0x00000000000014c8 <+5>:     mov    %rsp,%rbp
   0x00000000000014cb <+8>:     sub    $0x50,%rsp
   0x00000000000014cf <+12>:    mov    %rdi,-0x38(%rbp)
   0x00000000000014d3 <+16>:    mov    %esi,-0x3c(%rbp)
   0x00000000000014d6 <+19>:    mov    %rdx,-0x48(%rbp)
   0x00000000000014da <+23>:    movl   $0x0,-0x20(%rbp)
   0x00000000000014e1 <+30>:    movl   $0x0,-0x1c(%rbp)
   0x00000000000014e8 <+37>:    jmp    0x15ce <Shuffle+267>
   0x00000000000014ed <+42>:    mov    -0x48(%rbp),%rax
   0x00000000000014f1 <+46>:    mov    %rax,%rdi
   0x00000000000014f4 <+49>:    call   0x1434 <pop>
   0x00000000000014f9 <+54>:    mov    %al,-0x22(%rbp)
   0x00000000000014fc <+57>:    movsbl -0x22(%rbp),%eax
   0x0000000000001500 <+61>:    mov    %eax,-0x10(%rbp)
   0x0000000000001503 <+64>:    mov    -0x10(%rbp),%eax
   0x0000000000001506 <+67>:    mov    %eax,%edi
   0x0000000000001508 <+69>:    call   0x174b <fingerprint>
   0x000000000000150d <+74>:    mov    %eax,-0xc(%rbp)
   0x0000000000001510 <+77>:    mov    -0x20(%rbp),%edx
   0x0000000000001513 <+80>:    mov    -0x3c(%rbp),%ecx
   0x0000000000001516 <+83>:    mov    -0xc(%rbp),%eax
   0x0000000000001519 <+86>:    mov    %ecx,%esi
   0x000000000000151b <+88>:    mov    %eax,%edi
   0x000000000000151d <+90>:    call   0x16d7 <hash1>
   0x0000000000001522 <+95>:    mov    %eax,-0x8(%rbp)
   0x0000000000001525 <+98>:    mov    -0x20(%rbp),%edx
   0x0000000000001528 <+101>:   mov    -0x3c(%rbp),%ecx
   0x000000000000152b <+104>:   mov    -0xc(%rbp),%eax
   0x000000000000152e <+107>:   mov    %ecx,%esi
   0x0000000000001530 <+109>:   mov    %eax,%edi
   0x0000000000001532 <+111>:   call   0x1708 <hash2>
   0x0000000000001537 <+116>:   mov    %eax,-0x4(%rbp)
   0x000000000000153a <+119>:   mov    -0x8(%rbp),%eax
   0x000000000000153d <+122>:   cltq
   0x000000000000153f <+124>:   lea    0x0(,%rax,4),%rdx
   0x0000000000001547 <+132>:   mov    -0x38(%rbp),%rax
   0x000000000000154b <+136>:   add    %rdx,%rax
   0x000000000000154e <+139>:   mov    (%rax),%eax
   0x0000000000001550 <+141>:   cmp    $0xffffffff,%eax
   0x0000000000001553 <+144>:   jne    0x1570 <Shuffle+173>
   0x0000000000001555 <+146>:   mov    -0x8(%rbp),%eax
   0x0000000000001558 <+149>:   cltq
   0x000000000000155a <+151>:   lea    0x0(,%rax,4),%rdx
   0x0000000000001562 <+159>:   mov    -0x38(%rbp),%rax
   0x0000000000001566 <+163>:   add    %rax,%rdx
   0x0000000000001569 <+166>:   mov    -0xc(%rbp),%eax
   0x000000000000156c <+169>:   mov    %eax,(%rdx)
   0x000000000000156e <+171>:   jmp    0x15b8 <Shuffle+245>
   0x0000000000001570 <+173>:   mov    -0x4(%rbp),%eax
   0x0000000000001573 <+176>:   cltq
   0x0000000000001575 <+178>:   lea    0x0(,%rax,4),%rdx
   0x000000000000157d <+186>:   mov    -0x38(%rbp),%rax
   0x0000000000001581 <+190>:   add    %rdx,%rax
   0x0000000000001584 <+193>:   mov    (%rax),%eax
   0x0000000000001586 <+195>:   cmp    $0xffffffff,%eax
   0x0000000000001589 <+198>:   jne    0x15a6 <Shuffle+227>
   0x000000000000158b <+200>:   mov    -0x4(%rbp),%eax
   0x000000000000158e <+203>:   cltq
   0x0000000000001590 <+205>:   lea    0x0(,%rax,4),%rdx
   0x0000000000001598 <+213>:   mov    -0x38(%rbp),%rax
   0x000000000000159c <+217>:   add    %rax,%rdx
   0x000000000000159f <+220>:   mov    -0xc(%rbp),%eax
   0x00000000000015a2 <+223>:   mov    %eax,(%rdx)
   0x00000000000015a4 <+225>:   jmp    0x15b8 <Shuffle+245>
   0x00000000000015a6 <+227>:   movsbl -0x22(%rbp),%edx
   0x00000000000015aa <+231>:   mov    -0x48(%rbp),%rax
   0x00000000000015ae <+235>:   mov    %edx,%esi
   0x00000000000015b0 <+237>:   mov    %rax,%rdi
   0x00000000000015b3 <+240>:   call   0x147a <push>
   0x00000000000015b8 <+245>:   addl   $0x1,-0x1c(%rbp)
   0x00000000000015bc <+249>:   addl   $0x1,-0x20(%rbp)
   0x00000000000015c0 <+253>:   mov    -0x3c(%rbp),%edx
   0x00000000000015c3 <+256>:   mov    %edx,%eax
   0x00000000000015c5 <+258>:   add    %eax,%eax
   0x00000000000015c7 <+260>:   add    %edx,%eax
   0x00000000000015c9 <+262>:   cmp    %eax,-0x1c(%rbp)
   0x00000000000015cc <+265>:   jg     0x15df <Shuffle+284>
   0x00000000000015ce <+267>:   mov    -0x48(%rbp),%rax
   0x00000000000015d2 <+271>:   mov    0x8(%rax),%eax
   0x00000000000015d5 <+274>:   test   %eax,%eax
   0x00000000000015d7 <+276>:   jg     0x14ed <Shuffle+42>
   0x00000000000015dd <+282>:   jmp    0x15e0 <Shuffle+285>
   0x00000000000015df <+284>:   nop
   0x00000000000015e0 <+285>:   movl   $0x0,-0x18(%rbp)
   0x00000000000015e7 <+292>:   movl   $0x0,-0x14(%rbp)
   0x00000000000015ee <+299>:   jmp    0x163e <Shuffle+379>
   0x00000000000015f0 <+301>:   mov    -0x48(%rbp),%rax
   0x00000000000015f4 <+305>:   mov    %rax,%rdi
   0x00000000000015f7 <+308>:   call   0x1434 <pop>
   0x00000000000015fc <+313>:   mov    %al,-0x21(%rbp)
   0x00000000000015ff <+316>:   jmp    0x1605 <Shuffle+322>
   0x0000000000001601 <+318>:   addl   $0x1,-0x18(%rbp)
   0x0000000000001605 <+322>:   mov    -0x18(%rbp),%eax
   0x0000000000001608 <+325>:   cltq
   0x000000000000160a <+327>:   lea    0x0(,%rax,4),%rdx
   0x0000000000001612 <+335>:   mov    -0x38(%rbp),%rax
   0x0000000000001616 <+339>:   add    %rdx,%rax
   0x0000000000001619 <+342>:   mov    (%rax),%eax
   0x000000000000161b <+344>:   cmp    $0xffffffff,%eax
   0x000000000000161e <+347>:   jne    0x1601 <Shuffle+318>
   0x0000000000001620 <+349>:   mov    -0x18(%rbp),%eax
   0x0000000000001623 <+352>:   cltq
   0x0000000000001625 <+354>:   lea    0x0(,%rax,4),%rdx
   0x000000000000162d <+362>:   mov    -0x38(%rbp),%rax
   0x0000000000001631 <+366>:   add    %rax,%rdx
   0x0000000000001634 <+369>:   movsbl -0x21(%rbp),%eax
   0x0000000000001638 <+373>:   mov    %eax,(%rdx)
   0x000000000000163a <+375>:   addl   $0x1,-0x14(%rbp)
   0x000000000000163e <+379>:   mov    -0x48(%rbp),%rax
   0x0000000000001642 <+383>:   mov    0x8(%rax),%eax
   0x0000000000001645 <+386>:   cmp    %eax,-0x14(%rbp)
   0x0000000000001648 <+389>:   jl     0x15f0 <Shuffle+301>
   0x000000000000164a <+391>:   nop
   0x000000000000164b <+392>:   nop
   0x000000000000164c <+393>:   leave
   0x000000000000164d <+394>:   ret
```

printSlots:

```
   0x000000000000164e <+0>:     endbr64
   0x0000000000001652 <+4>:     push   %rbp
   0x0000000000001653 <+5>:     mov    %rsp,%rbp
   0x0000000000001656 <+8>:     sub    $0x20,%rsp
   0x000000000000165a <+12>:    mov    %rdi,-0x18(%rbp)
   0x000000000000165e <+16>:    mov    %esi,-0x1c(%rbp)
   0x0000000000001661 <+19>:    lea    0xbbe(%rip),%rax        # 0x2226
   0x0000000000001668 <+26>:    mov    %rax,%rdi
   0x000000000000166b <+29>:    mov    $0x0,%eax
   0x0000000000001670 <+34>:    call   0x1100 <printf@plt>
   0x0000000000001675 <+39>:    movl   $0x0,-0x4(%rbp)
   0x000000000000167c <+46>:    jmp    0x16b1 <printSlots+99>
   0x000000000000167e <+48>:    mov    -0x4(%rbp),%eax
   0x0000000000001681 <+51>:    cltq
   0x0000000000001683 <+53>:    lea    0x0(,%rax,4),%rdx
   0x000000000000168b <+61>:    mov    -0x18(%rbp),%rax
   0x000000000000168f <+65>:    add    %rdx,%rax
   0x0000000000001692 <+68>:    mov    (%rax),%eax
   0x0000000000001694 <+70>:    movsbl %al,%eax
   0x0000000000001697 <+73>:    mov    %eax,%esi
   0x0000000000001699 <+75>:    lea    0xb8a(%rip),%rax        # 0x222a
   0x00000000000016a0 <+82>:    mov    %rax,%rdi
   0x00000000000016a3 <+85>:    mov    $0x0,%eax
   0x00000000000016a8 <+90>:    call   0x1100 <printf@plt>
   0x00000000000016ad <+95>:    addl   $0x1,-0x4(%rbp)
   0x00000000000016b1 <+99>:    mov    -0x4(%rbp),%eax
   0x00000000000016b4 <+102>:   cmp    -0x1c(%rbp),%eax
   0x00000000000016b7 <+105>:   jl     0x167e <printSlots+48>
   0x00000000000016b9 <+107>:   mov    $0xa,%edi
   0x00000000000016be <+112>:   call   0x10c0 <putchar@plt>
   0x00000000000016c3 <+117>:   mov    -0x1c(%rbp),%edx
   0x00000000000016c6 <+120>:   mov    -0x18(%rbp),%rax
   0x00000000000016ca <+124>:   mov    %edx,%esi
   0x00000000000016cc <+126>:   mov    %rax,%rdi
   0x00000000000016cf <+129>:   call   0x186e <checkValid>
   0x00000000000016d4 <+134>:   nop
   0x00000000000016d5 <+135>:   leave
   0x00000000000016d6 <+136>:   ret
```

From these two functions, we can see hash1&hash2 in Shuffle and checkValid in printSlots. Ideally you would go to checkValid to see what the gate is looking for

4.Once you disassemble checkValid, you can go into the assembly and start going line by line. Here you can see on line <+76> that it will

printLose() if you are not equal to 6. So your input has to be 6 characters.

```
0x00005555555558ba <+76>:    cmpl   $0x6,-0x3c(%rbp)
0x00005555555558be <+80>:    je     0x5555555558cc <checkValid+94>
0x00005555555558c0 <+82>:    mov    $0x0,%eax
0x00005555555558c5 <+87>:    call   0x55555555540b <printLose>
0x00005555555558ca <+92>:    jmp    0x55555555591a <checkValid+172>
0x00005555555558cc <+94>:    movl   $0x0,-0x24(%rbp)
0x00005555555558d3 <+101>:   jmp    0x555555555908 <checkValid+154>
```

Then from there you can go through the assembly until you hit this section

```
0x00005555555558eb <+125>:   mov    -0x24(%rbp),%eax
0x00005555555558ee <+128>:   cltq
0x00005555555558f0 <+130>:   mov    -0x20(%rbp,%rax,4),%eax
0x00005555555558f4 <+134>:   cmp    %eax,%edx
0x00005555555558f6 <+136>:   je     0x555555555904 <checkValid+150>
0x00005555555558f8 <+138>:   mov    $0x0,%eax
0x00005555555558fd <+143>:   call   0x55555555540b <printLose>
0x0000555555555902 <+148>:   jmp    0x55555555591a <checkValid+172>
0x0000555555555904 <+150>:   addl   $0x1,-0x24(%rbp)
```

This is now comparing some section of code to your input and making sure that it is equal to the expected output. From here you can investigate each letter by letter and eventually find out that the program is looking for

[70][108][97][109][101][95] -> Flame_

You then have to go through Shuffle and investigate the hash functions

```
Dump of assembler code for function hash1:
   0x00005555555556d7 <+0>:     endbr64
   0x00005555555556db <+4>:     push   %rbp
   0x00005555555556dc <+5>:     mov    %rsp,%rbp
   0x00005555555556df <+8>:     mov    %edi,-0x14(%rbp)
   0x00005555555556e2 <+11>:    mov    %esi,-0x18(%rbp)
   0x00005555555556e5 <+14>:    mov    %edx,-0x1c(%rbp)
   0x00005555555556e8 <+17>:    mov    -0x14(%rbp),%eax
   0x00005555555556eb <+20>:    cltd
   0x00005555555556ec <+21>:    idivl  -0x18(%rbp)
   0x00005555555556ef <+24>:    mov    %edx,%eax
   0x00005555555556f1 <+26>:    sub    -0x1c(%rbp),%eax
   0x00005555555556f4 <+29>:    mov    %eax,-0x4(%rbp)
   0x00005555555556f7 <+32>:    jmp    0x5555555556fd <hash1+38>
   0x00005555555556f9 <+34>:    addl   $0x6,-0x4(%rbp)
   0x00005555555556fd <+38>:    cmpl   $0x0,-0x4(%rbp)
   0x0000555555555701 <+42>:    js     0x5555555556f9 <hash1+34>
   0x0000555555555703 <+44>:    mov    -0x4(%rbp),%eax
   0x0000555555555706 <+47>:    pop    %rbp
   0x0000555555555707 <+48>:    ret
```

This equates to (num % (length of input))

```
Dump of assembler code for function hash2:
   0x0000555555555708 <+0>:     endbr64
   0x000055555555570c <+4>:     push   %rbp
   0x000055555555570d <+5>:     mov    %rsp,%rbp
   0x0000555555555710 <+8>:     mov    %edi,-0x14(%rbp)
   0x0000555555555713 <+11>:    mov    %esi,-0x18(%rbp)
   0x0000555555555716 <+14>:    mov    %edx,-0x1c(%rbp)
   0x0000555555555719 <+17>:    mov    -0x18(%rbp),%eax
   0x000055555555571c <+20>:    lea    0x1(%rax),%esi
   0x000055555555571f <+23>:    mov    -0x14(%rbp),%eax
   0x0000555555555722 <+26>:    cltd
   0x0000555555555723 <+27>:    idiv   %esi
   0x0000555555555725 <+29>:    mov    %edx,%ecx
   0x0000555555555727 <+31>:    mov    -0x1c(%rbp),%eax
   0x000055555555572a <+34>:    mov    %eax,%edx
   0x000055555555572c <+36>:    shl    $0x2,%eax
   0x000055555555572f <+39>:    sub    %eax,%edx
   0x0000555555555731 <+41>:    lea    (%rcx,%rdx,1),%eax
   0x0000555555555734 <+44>:    sub    $0x1,%eax
   0x0000555555555737 <+47>:    mov    %eax,-0x4(%rbp)
   0x000055555555573a <+50>:    jmp    0x555555555740 <hash2+56>
   0x000055555555573c <+52>:    addl   $0x6,-0x4(%rbp)
   0x0000555555555740 <+56>:    cmpl   $0x0,-0x4(%rbp)
   0x0000555555555744 <+60>:    js     0x55555555573c <hash2+52>
   0x0000555555555746 <+62>:    mov    -0x4(%rbp),%eax
   0x0000555555555749 <+65>:    pop    %rbp
   0x000055555555574a <+66>:    ret
End of assembler dump.
```

This is equal to

(num % (length of string +1)) - (3 * placement of letter in string))-1

If it's less than 0, we repeatedly go add 6 until it becomes positive.

From there they have to go through and reverse engineer the placement of the letters in the input to get the hash to spell out Flame_

There is an additional section in shuffle where if it cannot use hashes, it simply places it to the first empty bucket.

They will have to compute it based on the ascii representations of the letters, and then they will come to find out the input is

emaFl which will become Flame

```
FlameGate$ ./FlameGate

Hello fair traveler.. This gate protects people from the terrors inside this forest.
To get into the forest, you have to solve the puzzle of the gate to prove
you are clever enough to not succumb to the forest and go mad. Try at your own risk..
Enter up to 10 characters (without spaces): _emaFl


        [F]     [l]     [a]     [m]     [e]     [_]


        **The gate sets ablaze and crumbles down before your eyes**

Well done traveler...Go ahead and enter the forest if you dare..

Here is your ticket:flame{G0ing_Cuck00}
```

Flag

flame{G0ing_Cuck00}